

# 수정하지 않은 프로그램을 메모리 절반에서 실행하기: 디스크에 쓰지 않는 무손실 압축 사용자 공간 페이징의 가능성과 한계

임현석<sup>○</sup>

한신대학교 공공인재빅데이터융합학

sunlim0926@hs.ac.kr

## Running Unmodified Programs in Half Their Memory: Feasibility and Limits of Disk-Free, Lossless Compressed User-Space Paging

Hyeonseok Im<sup>○</sup>

Hanshin University

### 요약

메모리가 작은 개인 기기에서 데이터 분석·영상 처리 프로그램은 메모리 부족으로 강제 종료되거나 운영체제 스왑으로 크게 느려진다. 본 논문은 프로그램을 고치지 않고 LD\_PRELOAD로 4MiB 이상의 모든 할당을 Linux userfaultfd 기반 사용자 공간 페이징에 넣고, 쉬는 메모리 조각을 디스크에 쓰지 않고 무손실 압축만으로 줄여 프로세스 전체 메모리 상한을 지키는 방식을 구현하고 사전 등록된 기준으로 평가하였다. 원래 최대 메모리의 1/2 한도에서 NumPy 영상 처리와 scikit-learn 분류는 결과가 비트 단위로 같게 완주하였고(그대로 실행하면 강제 종료), 영상 처리는 같은 한도의 OS 스왑보다 3.3배 빨랐다. 반면 무작위 접근이 많은 pandas 데이터프레임은 되읽기 폭주로, 압축 후에도 한도보다 큰 시뮬레이션 기록은 정보량 한계로 실패하였다. 이로 부터 디스크 없는 압축 페이징이 유효한 작업의 조건(순차적 접근, 충분한 압축률)을 제시한다.

### 1. 서론

8~16GB 메모리의 노트북에서 메모리보다 큰 작업을 돌리는 일반적인 방법은 운영체제 스왑이다. 스왑은 응용의 접근 형태를 모르고 4KiB 쪽 단위로 저장 장치에 쓰기 때문에 느리고 SSD 수명을 소모한다. 응용을 고쳐 데이터를 나눠 처리하면 빠르지만, 이미 작성된 분석 코드와 라이브러리를 모두 고치기는 어렵다.

본 논문은 다음 질문을 다룬다. **프로그램을 고치지 않고, 디스크에 쓰지 않고, 결과를 바꾸지 않으면서(무손실), 프로세스 전체 메모리를 정해진 상한 안에 두고 메모리 절반으로 실행할 수 있는 작업은 어떤 것인가?** 기여는 다음과 같다.

1. 할당 가로채기, userfaultfd 페이징, 무손실 압축 전용 내보내기, 프로세스 전체 상한을 결합한 구현.
2. 성격이 다른 수정하지 않은 프로그램 4종에 대한 사전 등록 평가(2종 성공, 2종 실패).
3. 실패 원인 분석을 통한 유효 조건 정리.

### 2. 관련 연구

커널 압축 스왑(zswap)[5]과 데이터센터의 투명 메모리 오프로딩 TMO[4]는 압축 메모리 계층을 두지만 시스템 전체 설정이 필요하고 프로세스 단위 상한을 보장하지 않는다. 사용자 공간 페이징인 UMap[2]과 ExtMem[1]은 userfaultfd로 응용이 페이징 정책을 정하게 하며, ExtMem은 LD\_PRELOAD로 수정 없는 프로그램에 적용한다. 그러나 두 방식 모두 데이터를 파일이나 저장 장치로 내보낸다.

malloc 가로채기로 힙을 다른 메모리에 두는 방식도 오래되었다[3]. 본 논문의 개별 기법은 모두 기존에 있으며, 본 논문은 저장 장치를 쓰지 않는 무손실 압축 전용 조합과 그 한계의 측정에 초점을 둔다.

### 3. 제안 방법

#### 3.1 개요

공유 라이브러리를 LD\_PRELOAD로 적재하여 큰 할당을 페이징 영역에 넣는다. 페이징은 한도를 넘으면 최근에 쓰지 않을 조각을 압축해 실제 메모리를 돌려주고, 다시 닿으면 풀어 채운다. 한도는 프로세스 전체 메모리를 기준으로 계속 조정한다.

#### 3.2 할당 가로채기

malloc, calloc, realloc, free, posix\_memalign, aligned\_alloc, memalign, malloc\_usable\_size를 가로챈다. 4MiB 이상은 페이징 영역에서, 작은 할당은 원래 할당기에서 준다. 따라서 NumPy 배열뿐 아니라 scipy·pandas 등 C 확장이 직접 잡는 큰 버퍼도 관리된다. 페이징 스레드 안의 할당(압축 라이브러리 등)은 스레드 지역 포시로 원래 할당기에 보내 재귀를 막는다.

#### 3.3 페이징

페이징 영역은 userfaultfd에 MISSING·쓰기 보호 모드로 등록된 익명 메모리다. 프로그램은 일반 읽기·쓰기를 하며, 처음 닿은 1MiB 조각은 0으로, 내보낸 조각은 압축본을 풀어 UFFDIO\_COPY로 채운다. 내보낼 때는 조각을 쓰기 보호

한 뒤 원소 크기 단위 바이트 서플과 `zstd[6]`로 압축하고 `MADV_DONTNEED`로 메모리를 돌려준다. 압축 이득이 15% 미만인 조각은 내보내지 않는다. 데이터는 디스크에 쓰지 않는다.

3.4 프로세스 전체 상한

프로세스 예산  $T$ 가 주어지면 페이지 한도를  $T - (\text{프로세스 RSS} - \text{페이지 보유량}) - \text{여유}$ 로 두고, 페이지 폴트마다 `/proc/self/statm`으로 다시 계산한다. 페이지 밖 메모리 (작은 할당, 인터프리터)가 늘면 페이지가 더 내보낸다.

3.5 구현상 문제

NumPy는 큰 배열에 `MADV_HUGEPAGE`를 요청하는데, `userfaultfd` 영역에서 이 요청 뒤 폴트가 페이지에 전달되지 않아 프로그램이 멈췄다. 페이지 영역에 `MADV_NOHUGEPAGE`를 걸고 그 영역에 대한 `MADV_HUGEPAGE` 요청을 무시하여 해결하였다.

4. 실험

4.1 환경과 작업

GitHub Actions Linux 실행기(커널 6.17, x86\_64 2 vCPU, 메모리 7.75GiB, 3GiB 스왑 파일, `zswap` 꺼짐)에서 Python 3.12, NumPy 2.5.3, pandas 3.0.6, scikit-learn 1.9.1로 측정하였다. 작업은 모두 결정적이며 결과의 SHA-256을 출력하고, 코드를 고치지 않고 실행하였다.

- **영상:** 512×512 16비트 영상 2,048장(1GiB)의 장별 평균, 문턱 화소 수, 배경 빼기, 전체 통계.
- **분류:** 문서 40만 × 단어 20만 희소 특징, scikit-learn `SGDClassifier` 5에폭.
- **데이터프레임:** pandas 1,600만 행 판매 표의 집계, 상위 100, 정렬.
- **시물레이션:** NumPy 4096×4096 float32 열 확산 300 단계, 10단계마다 장면 저장.

각 경우는 `cgroup` 메모리 한도(`MemoryMax`) 안에서 실행하였다. ① 한도 없이 그대로 실행해 최대 메모리  $P$ 를 채고, ② 한도  $L = P/2$  · 스왑 없이 그대로, ③ 제안 방법(한도  $L$ , 스왑 없음), ④ 한도  $L$ 에서 OS 스왑을 쓰고 그대로 실행하였다.

**사전 등록 기준**(실행 전에 고정, 결과를 본 뒤 바꾸지 않음):  $T_1$  ③의 출력이 ①과 같다,  $T_2$  ②는 끝나지 못한다,  $T_3$  ③이  $L$  안에서 끝나고 상한 초과(overrun)가 0회다. 제한 시간은 15분이며, 개발 측정에 근거해 영상 · 분류 성공, 데이터프레임 · 시물레이션 실패를 미리 예상으로 기록하였다.

4.2 메모리 절반에서 무엇이 끝나는가

표 1과 같이 영상과 분류는 그대로 실행하면 강제 종료되는 한도에서 결과가 같게 완주하였다.

표 1: 메모리 절반( $L = P/2$ )에서의 실행 시간(초). 영상 · 분류 통과, 나머지 실패(15분 제한)

| 작업      | P/L (MiB)   | ①    | ③ 제안 | ④ 스왑 |
|---------|-------------|------|------|------|
| 영상      | 1,578/788   | 5.2  | 21.2 | 69.5 |
| 분류      | 2,409/1,204 | 11.3 | 39.8 | 28.3 |
| 데이터 프레임 | 1,589/794   | 5.8  | >900 | 27.3 |
| 시물레이션   | 2,208/1,103 | 14.6 | >900 | 73.1 |

②는 네 작업 모두 메모리 부족으로 강제 종료되었다. 영상의 프로세스 최대 RSS는 726MiB로 한도 안이었고, 같은 한도의 OS 스왑보다 3.3배 빨랐다. 분류는 페이지 밖 메모리가 426MiB로 컸지만 프로세스 전체 상한이 페이지 뭉을 줄여 맞추었다. 다만 OS 스왑보다 1.4배 느렸는데, 매 에폭이 모든 조각을 다시 풀어야 하기 때문으로 본다.

4.3 왜 실패하는가

표 2: 제안 방법의 페이지 통계

| 작업      | 폴트      | 압축 전→후(GB)  | 압축비 | overrun |
|---------|---------|-------------|-----|---------|
| 영상      | 7,485   | 7.56→3.42   | 2.2 | 0       |
| 분류      | 9,842   | 8.02→1.53   | 5.3 | 0       |
| 데이터 프레임 | 195,123 | 202.8→69.1  | 2.9 | 0       |
| 시물레이션   | 371,804 | 361.7→161.6 | 2.2 | 279,314 |

**무작위 접근.** 데이터프레임의 정렬 · 그룹 집계는 메모리 곳곳을 몇 바이트씩 읽는다. 1MiB 조각 단위로는 몇 바이트를 위해 1MiB를 풀고 다시 압축하므로, 15분 동안 약 19.5만 번의 폴트와 203GB의 압축 입력이 생겼다. 조각을 64KiB로 줄인 개발 측정에서도 폴트가 약 206만 번으로 늘어 제한 시간을 넘었다. 같은 조건의 OS 스왑은 4KiB 단위로 27.3초에 끝났다.

**정보량 한계.** 시물레이션이 저장한 장면 30개(약 1.9GB)는 무손실로 약 42%만 줄어 압축본만 약 1.1GB이며, 프로세스 상한을 맞추려 줄어든 페이지 한도(약 1.0GB)보다 크다. 디스크에 쓰지 않는 무손실 방식으로는 원리적으로 들어갈 수 없으며, 27.9만 회의 overrun이 이를 보인다.

5. 결론

제안 방법은 (1) 접근이 순차적이거나 조각 단위 국소성이 있고 (2) 쉬는 데이터의 압축률이 메모리 부족분을 덮을 만큼 높은 작업에서, 프로그램을 고치지 않고 디스크에 쓰지 않으며 결과를 바꾸지 않고 메모리 절반으로 실행하게 한다. 영상 처리에서는 OS 스왑보다 3.3배 빨랐다. 무작위 접근에는 접근 형태에 맞춘 적응형 조각 크기, 비슷한 장면이 이어

지는 데이터에는 장면 간 차분 압축이 후속 과제다. 실험은 단일 실행기, 작업 4종, 각 1회 측정이며, userfaultfd가 없는 macOS · Windows에는 적용되지 않는다.

## 참고 문헌

- [1] S. Jalalian, S. Patel, M. Rezaei Hajidehi, M. Seltzer, A. Fedorova, “ExtMem: Enabling Application-Aware Virtual Memory Management for Data-Intensive Applications,” in Proceedings of the 2024 USENIX Annual Technical Conference, 2024.
- [2] Lawrence Livermore National Laboratory, “UMap: Enabling Application-driven Optimizations for Page Management,” arXiv:1910.07566, 2019.
- [3] PMDK, “libvmmalloc: general purpose volatile memory allocation library,” <https://pmem.io/pmdk/>
- [4] J. Weiner et al., “TMO: Transparent Memory Offloading in Datacenters,” in Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, 2022.
- [5] The Linux Kernel documentation, “zswap,” <https://docs.kernel.org/admin-guide/mm/zswap.html>
- [6] Y. Collet, M. Kucherawy, “Zstandard Compression and the ‘application/zstd’ Media Type,” RFC 8878, 2021.